

RESOLUTION NUMERIQUE



TP n°1

Algorithmes de base :

dichotomie, Newton, dérivation et intégration

Dichotomie, Newton, dérivation et intégration numériques

OBJECTIFS DU TP :

Dans le cadre du travail de l'ingénieur (modélisation, traitement de données, résolution d'équations comportementales, simplification de modèles, etc...) on peut être amenés à utiliser des outils de résolution et/ou simulation numérique.

Au-delà des simulateurs présents sur le marché (Matlab, Scilab, Solidworks, etc..), que l'on abordera en TP, il peut être très utile de coder soit même certaines problématiques classiques. Le langage **Python**, privilégié en CPGE est pour cela parfaitement adapté...

Dans ce cadre, le programme de SII a fixé les compétences suivantes :

- Solution d'un système stationnaire par la méthode de **dichotomie** et **Newton** ;
- **Dérivation** et **intégration numérique** ;
- Résolution d'un système linéaire par la méthode du **pivot de Gauss-Jordan** ;
- Résolution d'équations différentielles par la méthode du **schéma explicite d'Euler** ;
- Résoudre un problème en utilisant une solution d'**intelligence artificielle**.

Dans ce 1^{er} TP, nous nous limiterons aux deux premières problématiques...

Afin de gagner en efficacité dans nos algorithmes, nous utiliserons allègrement le module **Numpy**. De même, nous tracerons le plus souvent une image de notre travail à l'aide du module **Matplotlib**. On rappelle dans cette 1^{ère} section les éléments de base de ces modules...

1. RAPPELS DES ELEMENTS DE BASE DE MATPLOTLIB ET NUMPY

On suppose que l'on a effectué les commandes :

 `import matplotlib.pyplot as plt`

```
plt.clf() #efface le tracé précédent
plt.plot(X,Y,label='approchée',color='b') #affiche la solution approchée et précise la légende en bleu
plt.plot(X,Y2,label='exacte',color='r') #affiche la solution exacte et précise la légende en rouge
plt.legend() #affiche la legende approchée et exacte
plt.title('Reponse indicielle') #affiche le titre
plt.grid() #affiche la grille
plt.xlabel('$t$') #affiche t pour l'axe des abscisses
plt.show() #commande l'affichage
```

 `import numpy as np`

```
np.pi #permet d'obtenir la valeur de pi
A=np.array([1,2,3,4],float) #créait un tableau A à une dimension contenant 1,2,3 et 4 définit comme flotants
A[0] #renvoie ici le premier élément de A, soit 1.0
A[1:3] #renvoie les éléments de A compris entre 1 et 3, soit ici 2.,3.
A[0:4:2] #renvoie les éléments de A compris entre 0 et 4 avec un pas de deux, soit ici 1.,3.
A=np.array([[1,2],[1,2]],[[1,2],[1,2]]) #créait un tableau A à trois dimensions toutes de dimension deux
A=np.eye(3,3) #créait une matrice A diagonale de dimension 3*3
A=np.zeros((2,3)) #créait une matrice A nulle de dimension 2*3
A=np.ones((3,2)) #créait une matrice identité de dimension 3*2
A=np.arange(0,1,0.1) #créait une liste allant de 0 à 1 (exclus) tous les 0.1, soit dix éléments
A=np.linspace(1,4,6) #créait une liste de six éléments également répartis allant de 1 à 4 (inclus)
A=np.random.rand(n,p) #créait une matrice de taille (n,p) à coefficients aléatoires uniformes sur [0,1]
B=np.sin(A) #créait une matrice de dimension de A des sinus de A
np.shape(A) #renvoie la dimension de A, ici (n,p)
np.flatten(A) #renvoie une liste (une dimension) des éléments de A
np.reshape(A,p,n) #refond la matrice A(n,p) en matrice A(p,n) (correspond ici à la transposée)
np.transpose(A,p,n) #transpose la matrice A
f(A) #applique la fonction f à tous les éléments de A
```

Cette liste est bien entendu non exhaustive...

Vous pourrez retrouver plus de possibilité dans la documentation python !

2. SOLUTION PAR DICHOTOMIE OU NEWTON

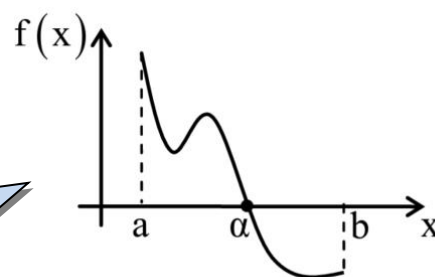
Commencez par créer un sous répertoire **nom_prénom** dans votre répertoire **psi/travail/info/SI**, dans lequel vous allez enregistrer vos programmes...

Problématique :

Soit f une fonction continue sur un intervalle $[a,b]$ de $\mathbb{R}$.

Nous cherchons par exemple $\alpha \in [a,b]$ tel que $f(\alpha) = 0$.

Ces méthodes peuvent bien entendu se généraliser à tout autre valeur de $f(\alpha)$ mais ne pourront s'appliquer que si α est unique dans l'intervalle $[a,b]$!



Critères d'arrêt :

Soit ε une tolérance fixée. On utilise, au choix :

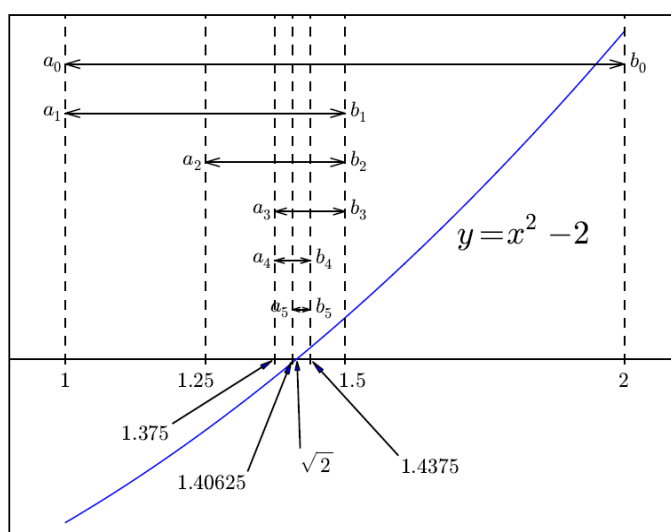
- un critère basé sur l'incrément : $|X_{n+1} - X_n| < \varepsilon$;
- un critère basé sur le résidu : $|f(X_n)| < \varepsilon$.

2.1 Principe de la méthode de dichotomie :

Dans le cas où l'on recherche un zéro, le principe de la dichotomie est d'encadrer de plus en plus finement un zéro possible, sa présence étant détectée par un changement de signe de la fonction, supposée continue.

Cette méthode est illustrée ci-contre...

Indice : La commande : `assert f(a)*f(b)<=0 and epsi>0` vérifie que la fonction change bien de signe entre a et b et que epsi est strictement positif...



Son principe de calcul est détaillé ci-dessous :

- On initialise l'encadrement par $a_0 < b_0$ telles que f change de signe entre ces valeurs ;
- On coupe l'intervalle en deux en son milieu. Puisqu'il y a changement de signe sur l'intervalle, il y a changement de signe sur l'une des deux moitiés. On conserve la moitié correspondante ;
- On continue de la sorte jusqu'à ce que l'intervalle obtenu soit suffisamment petit pour donner une valeur approchée du zéro à la marge de l'erreur **epsi** souhaitée.

Ecrire une fonction **sol_dicho(f, a, b, epsi)** qui renvoie l'approximation supérieure de la solution $f(x) = 0$ à **epsi** près.

Testez cette fonction !

On pourra par exemple tester avec la commande : `sol_dicho(lambda x : x**2-2,0,2,0.001)` qui renverra une approximation de $\sqrt{2}$ à 10^{-3} près.

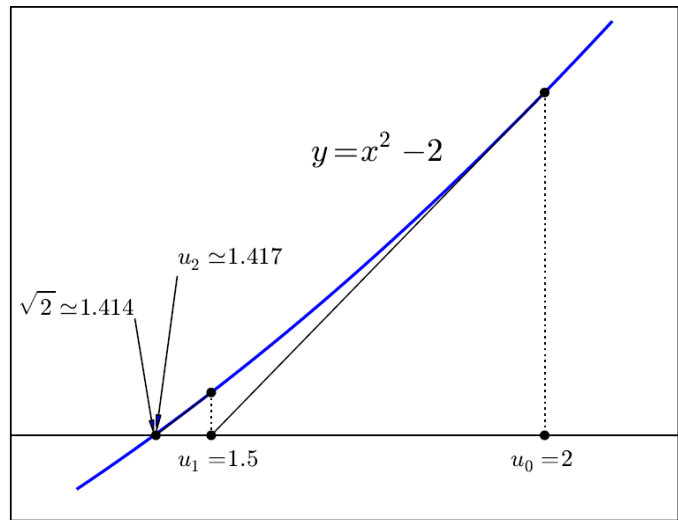
2.2 Principe de la méthode de Newton :

La méthode de Newton est bien plus efficace... à condition d'assurer sa convergence !

On suppose f dérivable. On part d'une valeur initiale u_0 . A partir de la valeur de $f(u_0)$, on suit la tangente orientée par $f'(u_0)$. L'intersection de cette tangente avec l'axe des abscisses donne une nouvelle valeur u_1 . On reprend la tangente depuis le point d'abscisse u_1 et ainsi de suite...

On a alors la suite : $u_{n+1} = u_n - \frac{f(u_n)}{f'(u_n)}$

Cette méthode est illustrée ci-contre...



Le principe du calcul est détaillé ci-dessous :

- On part de l'hypothèse que l'on connaît une fonction et sa dérivée ;
- On fixe une valeur initiale u_0 ;
- On construit u_1 comme l'intersection de la tangente en $f(u_0)$ et de l'axe des abscisses ;
- On réitère cette construction de façon à donner une valeur approchée du zéro à la marge de l'erreur ϵ souhaitée.

📄 Ecrire une fonction `sol_newton(f, g, u_0, epsi)` qui renvoie l'approximation supérieure de la solution $f(x) = 0$ à ϵ près.

📄 Testez cette fonction !

On pourra utiliser un test similaire à la solution par dichotomie et ainsi comparer les deux solutions...

🔒 Pensez à enregistrer ce programme sous le nom : **nom_prenom_zero**

2.3 Application : trouver le $T_{R5\%}$ d'un second ordre apériodique :

Afin de donner une illustration concrète à ces méthodes tout en restant dans des calculs simples, on se propose de trouver le temps de réponse à 5% pour un système du second ordre soumis à un échelon de Heaviside...

Pour mémoire, la réponse en Laplace peut alors s'écrire dans ce cas :

$$S(p) = \frac{K.e_0}{p(1 + \tau_1.p)(1 + \tau_2.p)} = \frac{K.e_0}{p(1 + (\tau_1 + \tau_2).p + \tau_1.\tau_2.p^2)}$$

Et la réponse temporelle est : $s(t) = K.e_0 \left(1 - \frac{1}{\tau_1 - \tau_2} \left(\tau_1.e^{-\frac{t}{\tau_1}} - \tau_2.e^{-\frac{t}{\tau_2}} \right) \right)$

Indice 1 : Partir d'une valeur approchée du $T_{R5\%} \approx 3(\tau_1 + \tau_2)$...

Indice 2 : L'intervalle pour la dichotomie pourra être fixé à 10% de $T_{R5\%}$ approché...

📄 Ecrire un script utilisant les fonctions précédentes, qui, après avoir demandé les valeurs de τ_1 , τ_2 et ϵ affiche les valeurs de $T_{R5\%D}$ (dichotomie) et $T_{R5\%N}$ (newton).

On prendra par ailleurs $K.e_0 = 1$ (pas d'incidence sur le $T_{R5\%}$).

🔒 Enregistrez ce programme sous le nom : **nom_prenom_tr5**

Indice 3 : On pourra se servir de Numpy...

3. DÉRIVATION OU INTÉGRATION NUMÉRIQUE

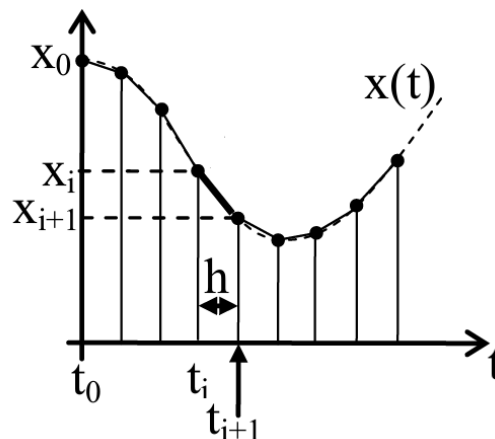
3.1 Dérivation numérique :

La méthode consiste à approximer une fonction par la méthode des accroissements finis. Pour ce faire, plusieurs approches sont possibles.

Soit pour un taux d'accroissement noté h :

- On utilise la valeur de la dérivée première arrière en t_k : $\frac{dx(t)}{dt} \simeq \frac{x_t - x_{t-h}}{h}$;
- On utilise la valeur de la dérivée première avant en t_k : $\frac{dx(t)}{dt} \simeq \frac{x_{t+h} - x_t}{h}$;
- On utilise la valeur de la dérivée seconde centrée en t_k : $\frac{d^2x(t)}{dt^2} \simeq \frac{x_{t+h} - 2x_t + x_{t-h}}{h^2}$;

On illustre ci-contre les valeurs prises par la dérivée première avant...



🖥 Ecrire une fonction **derivee_1_avant (f, a, h)** qui renvoie le nombre dérivé : $f'(a)$ pour un taux d'accroissement h .

🖥 Tester votre fonction.

On pourra par exemple tester avec la fonction : `lambda x : np.sin(x)` en 0 et en π .

🖥 Ecrire un script qui trace $f(t)$, $f'(t)$ sur l'intervalle $t \in a, b$ à partir de cette fonction.

🖥 Tester votre script.

On pourra par exemple tester avec la fonction : `lambda x : np.sin(x)` de 0 à 2π .

🖥 Compléter votre script avec une fonction **derivee_2 (f, a, h)** qui renvoie le nombre dérivé seconde : $f''(a)$ pour un même accroissement h .

🖥 Puis qui trace $f(t)$, $f'(t)$ et $f''(t)$ sur l'intervalle $t \in a, b$.

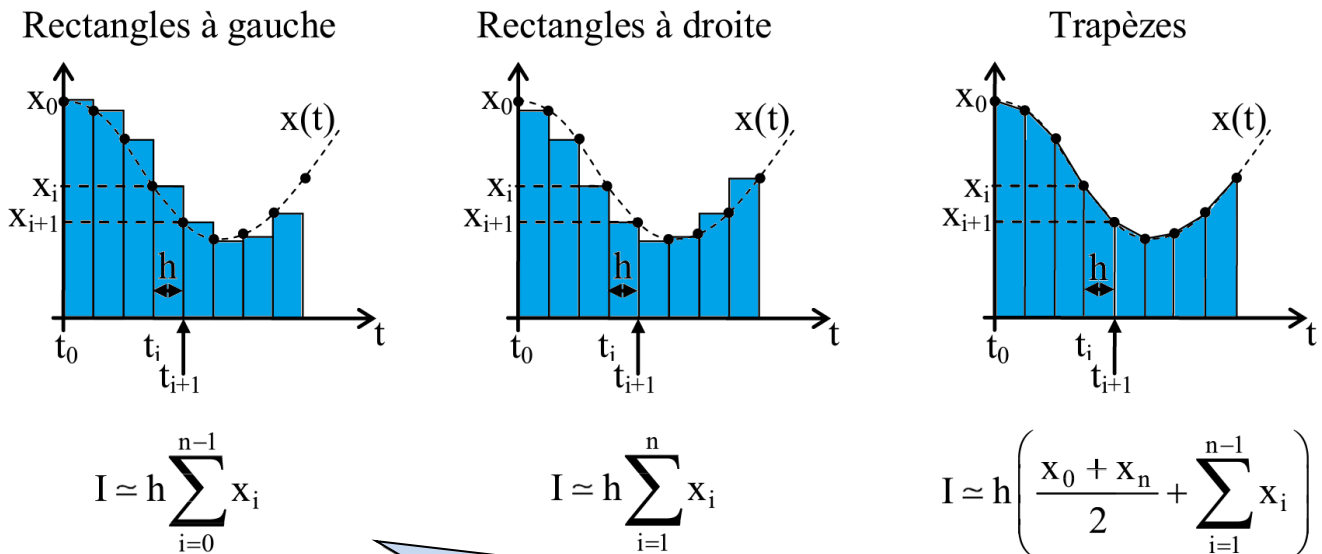
🖥 Tester votre nouveau script.

📁 Enregistrez votre script sous le nom : **nom_prenom_derive**

3.3 Intégration numérique :

On cherche à approximer l'intégrale $I = \int_a^b x(t)dt$.

La méthode générale consiste à calculer l'aire sous la courbe $x(t)$. Pour cela, plusieurs solutions s'offrent à nous :



Les deux premières correspondent à des sommes de Riemann...

Commencer par créer une fonction `rectangleg(f,a,b,n)` qui permet de calculer l'approximation par la gauche de : $\int_a^b f(t)dt$ par une somme de `n` rectangles.

Testez cette fonction avec une fonction connue (sinus par exemple) en faisant varier le nombre `n` de rectangles.

Créez dans le même `script` deux autres fonctions `rectangled(f,a,b,n)` et `trapeze(f,a,b,n)` qui calculent respectivement l'approximation de : $\int_a^b f(t)dt$ par la droite et par des trapèzes.

Testez ce `script` en comparant les résultats renvoyés par ces trois fonctions.

Enregistrez ce programme sous le nom : **nom_prénom_tp6_prog5**

PARTIE OPTIONNELLE

S'il vous reste du temps, vous pouvez essayer d'illustrer les sommes de Riemann et/ou la méthode des trapèzes en reproduisant les illustrations ci-dessus...